System Specification: 8-bit Parallel Bus (2 Masters, 3 Slaves)

Oct 21, 2026 · @Lux

1. Introduction

This document is the single source of truth for building and verifying an 8-bit parallel system bus with two masters and three slaves (4 KB, 4 KB and 2 KB), implemented on Altera DE0 boards. RTL designers use it to code each block; verification engineers use it to write testbenches and pass/fail checks without asking the designers.

1.1 Scope

- **In scope:** priority arbiter, address decoder, top-level bus interconnect (muxing, reset, clock), synthesizable master and slave models, DE0 demo wrapper, and the verification plan for all of them.
- **Out of scope:** burst transfers, split/pipelined transactions, bus error recovery beyond a flag, DMA, caches, and any clock other than the 50 MHz board clock.

1.2 Source requirements (EN5102 Assignment 2, Dr. S. Thayaparan)

Item	Requirement from the brief
Bus width	8-bit parallel data bus
Topology	2 masters, 3 slaves; slave memory spaces 4K, 4K, 2K
RTL style	Asynchronous reset, positive-edge clock
Hardware	Bus design on one DE0 board; masters and slave models on a second DE0 board
Arbiter	Priority-based arbitration; verify idle, single-master request, two-master request
Decoder	Verify address decoding for 3 slaves and the address mapping

Item	Requirement from the brief
Top level	Verify reset, idle, one master request, two master requests
Demo controls	One push-button = reset; 7-segment shows mode (1 idle, 2 one master, 3 two masters); 4 switches = Address[3:0]; 4 switches = Data[3:0]; upper bits 0
Deliverables	Report (20 marks): functional description, IO definitions, timing diagrams, RTL and testbench code. Demo (20 marks). Deadline: 15 days from issue

1.3 Source-document discrepancy

The brief's title slide reads "Assignment 2" but the guidelines slide reads "Assignment 3 - Guidelines". This spec treats the guidelines as belonging to this assignment (they describe the same DE0 demo). Confirm with the instructor; see Section 13.

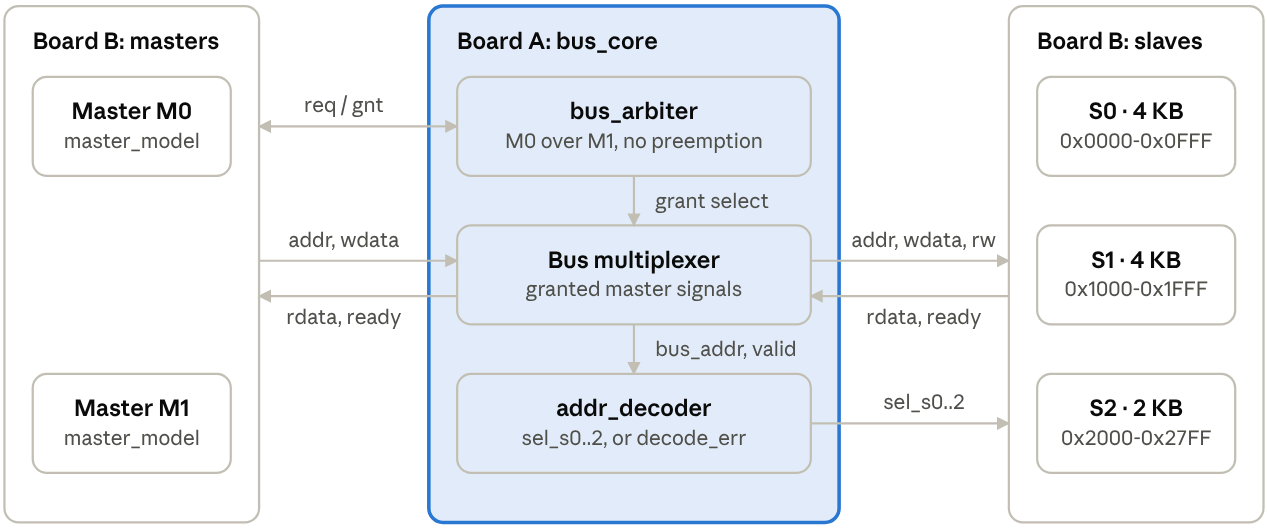
1.4 Conventions

- **SHALL** = mandatory and testable; **SHOULD** = recommended; **MAY** = optional.
- Requirement IDs (REQ-xx) are used in the verification plan for traceability.
- Signal names are lowercase with a suffix: `_n` active-low, `_i` input, `_o` output, `_q` registered. Parameters and constants are UPPERCASE.
- Numbers: `0x` prefix = hexadecimal; `4'b` style = Verilog binary literals. Clock is 50 MHz (20 ns period) unless stated.
- Items marked **[ASSUMPTION]** are design decisions made here where the brief is silent; they are consolidated in Section 13 for sign-off.

2. System overview

Two bus masters (M0, M1) share one 8-bit data path to three memory slaves (S0, S1, S2). A priority arbiter grants the bus to one master at a time, and an address decoder turns the granted master's address into a single slave select. The bus fabric (arbiter, decoder, multiplexers) lives on DE0 board A; the master and slave models live on DE0 board B.

All master-slave traffic passes through bus_core on board A



system block diagram · 2 masters, bus_core, 3 slaves

Board B holds both masters and all three slaves. Board A holds arbitration, address decoding and the multiplexers. The unmapped-address responder (Section 7.3) sits inside `bus_core` and is not drawn.

2.1 Components

Block	Board	Function
`bus_arbiter`	A	Priority arbitration of M0 and M1; registered one-hot grants
`addr_decoder`	A	Combinational decode of `bus_addr` into `sel_s0..sel_s2` plus an unmapped flag
`bus_fabric` (top-level glue)	A	Master-to-bus multiplexers, slave-to-master read-data and ready multiplexers, reset synchronisation
`demo_ctrl`	A or B	Reads switches and buttons, drives the 7-segment mode display
`master_model` x2	B	Issues read/write transactions when its request button or script fires
`slave_mem` x3	B	Synchronous RAM with a one-wait-state ready handshake (4 KB, 4 KB, 2 KB)

2.2 Key parameters

Parameter	Value	Note
`DATA_W`	8 bits	Fixed by the brief
`ADDR_W`	16 bits	[ASSUMPTION] Gives room above the 10 KB mapped space; 14 bits would suffice
`N_MASTERS`	2	M0 has higher priority (see Section 5)
`N_SLAVES`	3	S0 4 KB, S1 4 KB, S2 2 KB
Clock	50 MHz, `clk`	DE0 on-board oscillator, positive-edge; in the two-board demo the bus clock is 10 MHz (Section 10.3)
Reset	`rst_n`, active low	Asynchronous assert, synchronous de-assert [ASSUMPTION]

2.3 Memory map

Slave	Size	Address range	Decode condition	Local address
S0	4 KB	`0x0000` - `0x0FFF`	`A[15:12] == 4'h0`	`A[11:0]`
S1	4 KB	`0x1000` - `0x1FFF`	`A[15:12] == 4'h1`	`A[11:0]`
S2	2 KB	`0x2000` - `0x27FF`	`A[15:11] == 5'b00100`	`A[10:0]`
Unmapped	rest	`0x2800` - `0xFFFF`	none of the above	n/a

A read of an unmapped address SHALL return `0x00` and raise `decode_err`; a write SHALL be discarded and also raise `decode_err` (Section 6).

3. Requirements

Every requirement below is testable and maps to at least one test case in Section 11. Test IDs use the prefixes ARB, DEC and TOP.

ID	Requirement	Verified by
REQ-SYS-01	The data path SHALL be 8 bits wide (`DATA_W = 8`).	TOP-03
REQ-SYS-02	All sequential logic SHALL use `posedge clk` and asynchronous active-low `rst_n`.	ARB-01, ARB-09, TOP-01, TOP-07
REQ-SYS-03	At most one master SHALL own the bus in any clock cycle.	ARB-10, TOP-04
REQ-SYS-04	After reset the bus SHALL be idle: no grant, no slave select, `valid` ignored.	ARB-01, TOP-01
REQ-ARB-01	Arbitration SHALL be fixed-priority: M0 outranks M1.	ARB-05
REQ-ARB-02	With an idle bus and a single request, the grant SHALL be asserted exactly 1 clock after the request is first sampled high.	ARB-03, ARB-04

ID	Requirement	Verified by
REQ-ARB-03	Arbitration SHALL be non-preemptive: a granted master keeps the bus until it drops its request.	ARB-06
REQ-ARB-04	When the owner drops its request and the other master is requesting, the grant SHALL move directly to the other master with no overlap.	ARB-07, ARB-08
REQ-ARB-05	With no request, both grants SHALL be 0. Grants SHALL never both be 1.	ARB-02, ARB-10
REQ-DEC-01	The decoder SHALL assert exactly one of `sel_s0`, `sel_s1`, `sel_s2` for a mapped address while the bus is active, per the Section 2.3 map.	DEC-01 to DEC-03, DEC-06
REQ-DEC-02	For an unmapped address, no slave select SHALL be asserted and `decode_err` SHALL be 1.	DEC-04
REQ-DEC-03	When the bus is not active (`bus_valid = 0`), all selects and `decode_err` SHALL be 0.	DEC-05
REQ-BUS-01	A transfer SHALL complete only when the selected slave asserts `ready`; the master SHALL hold address, data and `rw` stable until then.	TOP-03
REQ-BUS-02	Each slave SHALL respond with `ready` exactly 1 clock after its select and `valid` are sampled (one wait state).	TOP-03
REQ-BUS-03	A write SHALL update only the addressed slave's memory; all other memory contents SHALL be unchanged.	TOP-08
REQ-BUS-04	A read SHALL return the last value written to that address, and the initial value `0x00` if never written since FPGA configuration (reset does not clear the RAM).	TOP-03, TOP-08

ID	Requirement	Verified by
REQ-BUS-05	An access to an unmapped address SHALL still complete (the fabric asserts `ready` with `decode_err`) so the master never hangs.	TOP-05
REQ-DEMO-01	One push-button SHALL drive `rst_n`.	TOP-09
REQ-DEMO-02	The 7-segment display SHALL show 1 (idle), 2 (one master request) or 3 (two master requests).	TOP-09
REQ-DEMO-03	Four switches SHALL set `Address[3:0]` and four switches SHALL set `Data[3:0]`; the upper bits SHALL be 0.	TOP-09
REQ-NF-01	Each board's logic SHALL close timing at 50 MHz on the Cyclone III EP3C16F484 with at least 2 ns of slack, and the inter-board link SHALL meet its constrained setup and hold timing at the 10 MHz demo bus clock (Section 10.3).	Fitter report
REQ-NF-02	RTL SHALL be synthesizable Verilog-2001 or SystemVerilog with no latches and no combinational loops.	Lint, fitter report

4. Bus protocol and signal definitions

The bus is a synchronous, single-transfer, request/grant protocol with a ready handshake. A transfer takes 4 clocks from request to completion on an idle bus. All signals are sampled on `posedge clk`. In this section "cycle N" means the interval after rising edge N.

4.1 Master port (one per master, `x` = 0 or 1)

Signal	Dir (master)	Width	Description
`mx_req`	out	1	Bus request. Held high from request until the transfer completes.
`mx_gnt`	in	1	Bus grant from the arbiter. Registered.

Signal	Dir (master)	Width	Description
`mx_valid`	out	1	Transfer valid. SHALL only be high while `mx_gnt` is high.
`mx_rw`	out	1	1 = write, 0 = read. Stable while `mx_valid` is high.
`mx_addr`	out	16	Byte address. Stable while `mx_valid` is high.
`mx_wdata`	out	8	Write data. Stable while `mx_valid` is high.
`mx_rdata`	in	8	Read data. Valid in the cycle `mx_ready` is high.
`mx_ready`	in	1	Transfer complete (1-cycle pulse), gated by `mx_gnt`.
`mx_err`	in	1	Qualifies `mx_ready`: 1 means the address was unmapped.

4.2 Slave port (one per slave, `y` = 0, 1 or 2)

Signal	Dir (slave)	Width	Description
`sy_sel`	in	1	Slave select from the decoder (already qualified by `bus_valid`).
`sy_rw`	in	1	Shared `bus_rw`.
`sy_addr`	in	12 / 12 / 11	Local address: `bus_addr[11:0]` (S0, S1) or `bus_addr[10:0]` (S2).
`sy_wdata`	in	8	Shared `bus_wdata`.
`sy_rdata`	out	8	Read data, registered. Valid while `sy_ready` is high.
`sy_ready`	out	1	Access done, registered 1-cycle pulse.

4.3 Shared bus (internal to the fabric)

`bus_valid`, `bus_rw`, `bus_addr[15:0]` and `bus_wdata[7:0]` are the granted master's signals, selected by a one-hot AND-OR multiplexer driven by `m0_gnt` and `m1_gnt`. When no grant is active they all read 0. `bus_rdata[7:0]`, `bus_ready` and `bus_err` flow back from the slaves and the unmapped-address responder.

4.4 Protocol rules

1. A master asserts `mx_req` and waits for `mx_gnt`.
2. On the first edge where it samples `mx_gnt = 1`, the master drives `mx_valid`, `mx_rw`, `mx_addr` and `mx_wdata` (registered outputs).
3. The decoder raises exactly one select. The slave samples `sel = 1` and `ready = 0`, performs the access, and registers `ready = 1` for one cycle. A write updates memory at that edge; a read loads `rdata` at that edge.
4. The master samples `mx_ready = 1`, captures `mx_rdata` if reading, then drops `mx_valid` and `mx_req` together.
5. The slave clears `ready` on the next edge (`ready <= sel & ~ready`), so a held `valid` cannot trigger a second access.
6. A master SHALL NOT drop `mx_req` before `mx_ready`. Doing so is a protocol violation flagged by the checkers (Section 11).
7. A master SHALL keep `mx_req` low for at least 1 cycle between transactions, which bounds the other master's wait to one transfer.
8. A master MAY issue several transfers under one grant by pulsing `mx_valid` again after each `mx_ready`, as long as `mx_req` stays high. [ASSUMPTION] The baseline demo uses one transfer per grant.
9. Unmapped access: the fabric's responder registers `bus_ready = 1` and `bus_err = 1` one cycle after `bus_valid` with `decode_err`, using the same rule as rule 5. Read data is `0x00`; writes are discarded.

5. Arbiter specification (`bus_arbiter`)

The arbiter is a 3-state Moore FSM with registered one-hot grants. M0 has fixed priority over M1, but only at the moment the bus is free; a granted master is never preempted.

5.1 Ports

Port	Dir	Width	Description
`clk`	in	1	50 MHz system clock, positive edge
`rst_n`	in	1	Asynchronous active-low reset
`m0_req`	in	1	Request from master 0 (higher priority)

Port	Dir	Width	Description
`m1_req`	in	1	Request from master 1
`m0_gnt`	out	1	Grant to master 0 (registered, directly a state bit)
`m1_gnt`	out	1	Grant to master 1 (registered, directly a state bit)
`bus_active`	out	1	`m0_gnt | m1_gnt`; status output for LEDs and debug (the decoder is qualified by bus_valid, not by this signal)
`mode`	out	2	Demo mode from the raw requests: `2'd1` none, `2'd2` one, `2'd3` both. Combinational

5.2 State table

One-hot encoding, `state = {G1, G0, IDLE}`. Outputs depend on state only (Moore).

Current state	`m0_req`	`m1_req`	Next state	Outputs after the edge (`m0_gnt`, `m1_gnt`)
IDLE	1	x	G0	1, 0
IDLE	0	1	G1	0, 1
IDLE	0	0	IDLE	0, 0
G0	1	x	G0	1, 0
G0	0	1	G1	0, 1
G0	0	0	IDLE	0, 0
G1	x	1	G1	0, 1
G1	1	0	G0	1, 0
G1	0	0	IDLE	0, 0

Asynchronous reset forces `IDLE`, so both grants fall to 0 without waiting for a clock edge.

5.3 Behavioural rules

- **Mutual exclusion:** `m0_gnt` and `m1_gnt` are never both 1 (one-hot or zero state).

- **Grant latency:** 1 clock from the first sampled request on an idle bus.
- **Release latency:** 1 clock from the owner dropping its request.
- **Simultaneous requests from IDLE:** M0 wins. M1 keeps `m1_req` high and receives the bus when M0 drops `m0_req`.
- **No preemption:** if M1 owns the bus and M0 requests, M0 waits until `m1_req` falls.
- **Direct handover:** the G0 to G1 and G1 to G0 transitions skip IDLE, so no bus cycle is wasted, and the grants still never overlap because both are registered from one state.
- **Bounded wait:** with rule 7 of Section 4.4 (masters drop `req` between transactions), a waiting master is served after at most one transfer of the other master. A master that holds `req` high forever can starve the other; this is a protocol violation, not an arbiter fault.
- **Glitchy request:** a 1-cycle request pulse yields a 1-cycle grant. The arbiter does not filter it; the checker flags it as a master protocol violation.

5.4 Implementation notes for developers

- Use a single `always @(posedge clk or negedge rst_n)` block for state, and a separate combinational `always @*` for next state with a default assignment to avoid latches.
- Drive `m0_gnt` and `m1_gnt` straight from the state flops. Do not add combinational logic on the outputs.
- `mode` is not registered: `mode = (m0_req & m1_req) ? 2'd3 : (m0_req | m1_req) ? 2'd2 : 2'd1`. If the requests come from board B pins, synchronise them with a 2-flop synchroniser **before** they enter the FSM (Section 8.4).

6. Address decoder specification (`addr_decoder`)

The decoder is purely combinational. It converts the shared bus address into one slave select, and flags addresses that no slave owns.

6.1 Ports

Port	Dir	Width	Description
`bus_valid`	in	1	Shared bus valid (the granted master's `valid`; 0 when no grant)

Port	Dir	Width	Description
`bus_addr`	in	16	Shared bus address
`sel_s0`	out	1	Select for slave 0
`sel_s1`	out	1	Select for slave 1
`sel_s2`	out	1	Select for slave 2
`decode_err`	out	1	Address is unmapped while `bus_valid` is 1

6.2 Decode equations

```
hit_s0     = (bus_addr[15:12] == 4'h0)        // 0x0000 - 0x0FFF
hit_s1     = (bus_addr[15:12] == 4'h1)        // 0x1000 - 0x1FFF
hit_s2     = (bus_addr[15:11] == 5'b00100)    // 0x2000 - 0x27FF

sel_s0     = bus_valid & hit_s0
sel_s1     = bus_valid & hit_s1
sel_s2     = bus_valid & hit_s2
decode_err = bus_valid & ~(hit_s0 | hit_s1 | hit_s2)
```

The three `hit` ranges do not overlap, so at most one select is high. Selects and `decode_err` are mutually exclusive, and all four outputs are 0 when `bus_valid = 0`. Base addresses and sizes SHOULD be `localparam` values so the map can change in one place.

6.3 Reference vectors (testers: use these as directed tests)

`bus_addr`	`bus_valid`	Expected `sel_s0 / s1 / s2`	`decode_err`	Why
`0x0000`	1	1 / 0 / 0	0	First byte of S0
`0x0FFF`	1	1 / 0 / 0	0	Last byte of S0
`0x1000`	1	0 / 1 / 0	0	First byte of S1
`0x1FFF`	1	0 / 1 / 0	0	Last byte of S1
`0x2000`	1	0 / 0 / 1	0	First byte of S2
`0x27FF`	1	0 / 0 / 1	0	Last byte of S2

`bus_addr`	`bus_valid`	Expected `sel_s0 / s1 / s2`	`decode_err`	Why
`0x2800`	1	0 / 0 / 0	1	First address past S2
`0x3000`	1	0 / 0 / 0	1	Unmapped
`0x8000`	1	0 / 0 / 0	1	Unmapped, MSB set
`0xFFFF`	1	0 / 0 / 0	1	Top of address space
any	0	0 / 0 / 0	0	Bus not valid: everything quiet

7. Master and slave model specification

The models are synthesizable so they can run on DE0 board B. The simulation testbenches MAY also use behavioural bus-functional tasks, but they SHALL obey the same protocol (Section 4.4).

7.1 Master model (`master_model`)

A 4-state FSM that performs one transfer per `start` pulse.

Port	Dir	Width	Description
`clk` , `rst_n`	in	1	Clock and async active-low reset
`start`	in	1	Level or pulse from a debounced button; a rising edge starts one transfer
`cmd_rw`	in	1	1 = write, 0 = read; latched at start
`cmd_addr`	in	16	Address; latched at start
`cmd_wdata`	in	8	Write data; latched at start
`busy`	out	1	High from start until the transfer is done
`done`	out	1	1-cycle pulse when the transfer completes

Port	Dir	Width	Description
`rdata_q`	out	8	Last captured read data; held until the next read
`err_q`	out	1	Last transfer hit an unmapped address
`req`, `gnt`, `valid`, `rw`, `addr`, `wdata`, `rdata`, `ready`, `err`	bus side	see 4.1	Master port signals of Section 4.1

State	Outputs	Next state
`M_IDLE`	`req = 0`, `valid = 0`, `busy = 0`	`M_REQ` on a `start` rising edge (latch `cmd_*`)
`M_REQ`	`req = 1`, `valid = 0`	`M_XFER` when `gnt = 1`
`M_XFER`	`req = 1`, `valid = 1`, `rw`, `addr`, `wdata` driven	`M_DONE` when `ready = 1` (capture `rdata`, `err`)
`M_DONE`	`req = 0`, `valid = 0`, `done = 1`	`M_IDLE` (always, after 1 cycle)

Because `M_DONE` always lasts one cycle with `req = 0`, the master meets protocol rule 7 (at least 1 idle cycle between requests). A `start` received outside `M_IDLE` is ignored.

`HOLD` parameter (demo build, `HOLD = 1`). In `M_DONE` the master keeps `req = 1` instead of dropping it, and moves to a fifth state `M_HOLD` (`req = 1`, `valid = 0`) if the `start` input is still high. It leaves `M_HOLD` for `M_IDLE` when `start` falls, which drops `req`. This keeps the request, and therefore the 7-segment mode and the grant LEDs, visible for as long as the button is held, and it is allowed by protocol rule 8. The default is `HOLD = 0`, and all simulation regressions use the default.

7.2 Slave model (`slave_mem`)

A parameterised synchronous RAM with a one-wait-state ready handshake. Instantiate three times: `ADDR_W = 12` (S0), `12` (S1) and `11` (S2).

Port	Dir	Width	Description
`clk`, `rst_n`	in	1	Clock and async active-low reset

Port	Dir	Width	Description
`sel`	in	1	Slave select from the decoder
`rw`	in	1	1 = write
`addr`	in	`ADDR_W`	Local address
`wdata`	in	8	Write data
`rdata`	out	8	Registered read data
`ready`	out	1	Registered 1-cycle completion pulse

Behaviour, evaluated at every `posedge clk`:

```
ready <= sel & ~ready;
if (sel & ~ready) begin
    if (rw) mem[addr] <= wdata;
    else    rdata     <= mem[addr];
end
```

`rst_n` clears `ready` and `rdata` only. The memory array is inferred as Cyclone III M9K block RAM, which cannot be reset by `rst_n`; it is initialised to `0x00` at FPGA configuration through an `initial` block or a MIF file. The three slaves total 10 KB (about 80 Kbit), well inside the device's embedded memory, so no external SDRAM is needed.

7.3 Unmapped-address responder (in the fabric)

The responder is a 1-bit register: `unmap_ready <= decode_err & ~unmap_ready`. The fabric ORs it into `bus_ready`, drives `bus_err = decode_err & unmap_ready`, and forces `bus_rdata = 0x00` for that transfer, so a master never hangs on a bad address (REQ-BUS-05).

8. Top-level integration

The verified design unit is `bus_core`: arbiter, decoder, bus multiplexers and the unmapped-address responder, with no board-specific logic. Board wrappers (Section 10) add pins, synchronisers and the demo shim around it, so the same `bus_core` runs in simulation and on the DE0.

8.1 Module hierarchy

```
bus_core                  <- unit under test for all RTL testbenches
  bus_arbiter             (Section 5)
  addr_decoder            (Section 6)
  [AND-OR bus muxes]      (8.3)
  [unmapped responder]    (7.3)
bus_top_a                 <- DE0 board A: bus_core + link pins + demo shim +
7-seg
bus_top_b                 <- DE0 board B: master_model x2 + slave_mem x3 +
read mux + switches/buttons
tb_arbiter, tb_decoder, tb_bus_core   <- testbenches (Section 11)
```

8.2 `bus_core` ports

For `x` in 0, 1 and `y` in 0, 1, 2. Directions are from `bus_core`'s point of view, so they are the mirror of the Section 4.1 and 4.2 tables.

Port group	Dir	Width	Notes
`clk`, `rst_n`	in	1 each	Single clock domain; `rst_n` is already synchronised by the wrapper
`mx_req`, `mx_valid`, `mx_rw`	in	1 each	Per master
`mx_addr`	in	`ADDR_W`	Per master
`mx_wdata`	in	`DATA_W`	Per master
`mx_gnt`, `mx_ready`, `mx_err`	out	1 each	Per master; `mx_ready` and `mx_err` are gated by `mx_gnt`
`mx_rdata`	out	`DATA_W`	Per master (same value `bus_rdata`)
`sy_sel`, `sy_rw`	out	1 each	Per slave
`sy_addr`	out	12 / 12 / 11	Per slave, local address
`sy_wdata`	out	`DATA_W`	Per slave
`sy_rdata`	in	`DATA_W`	Per slave
`sy_ready`	in	1	Per slave

Port group	Dir	Width	Notes
`mode`	out	2	From the arbiter (Section 5.1)
`bus_active`, `decode_err`	out	1 each	Debug and LED outputs

Parameters: `ADDR_W = 16`, `DATA_W = 8`. Changing them is allowed; the memory map in Section 2.3 is fixed.

8.3 Interconnect equations

```
bus_valid = (m0_gnt & m0_valid) | (m1_gnt & m1_valid)
bus_rw    = (m0_gnt & m0_rw)    | (m1_gnt & m1_rw)
bus_addr  = ({ADDR_W{m0_gnt}} & m0_addr)  | ({ADDR_W{m1_gnt}} & m1_addr)
bus_wdata = ({DATA_W{m0_gnt}} & m0_wdata) | ({DATA_W{m1_gnt}} & m1_wdata)

// decoder: sel_s0..2, decode_err from bus_valid, bus_addr   (Section 6)

unmap_ready <= decode_err & ~unmap_ready              // registered
bus_ready  = s0_ready | s1_ready | s2_ready | unmap_ready
bus_err    = decode_err & unmap_ready
bus_rdata  = ({DATA_W{sel_s0}} & s0_rdata) | ({DATA_W{sel_s1}} & s1_rdata)
           | ({DATA_W{sel_s2}} & s2_rdata)            // 0x00 when unmapped

mx_ready   = bus_ready & mx_gnt
mx_err     = bus_err   & mx_gnt
mx_rdata   = bus_rdata
sy_sel / sy_rw / sy_addr / sy_wdata = decoder select / bus_rw / bus_addr[11:0
or 10:0] / bus_wdata
```

Because the master `valid` is ANDed with its own grant, a master that raises `valid` without a grant cannot reach the bus. The checkers rely on this and also flag it as a master protocol violation.

8.4 Clock and reset

- **One clock domain.** Everything in `bus_core`, both wrappers and both boards uses the same `clk` (Section 10.3 explains how it is shared).

- **Reset:** `rst_n` asserts asynchronously and releases synchronously, through a 2-flop reset synchroniser in each board wrapper. Every flop in `bus_core`, `master_model` and `slave_mem` uses `always @(posedge clk or negedge rst_n)`.
- **Asynchronous inputs** (push-buttons, slide switches, and all signals arriving over the inter-board link) SHALL pass through a 2-flop synchroniser in the wrapper before reaching any logic, except where Section 10.3 says a signal is sampled directly under a timing constraint.

8.5 Transfer latency (idle bus, one master)

Cycle	Event	Where
0	`mx_req` = 1	Master
1	`mx_gnt` = 1	Arbiter state flop
2	`mx_valid`, `rw`, `addr`, `wdata` driven; `sel_sy` high	Master, decoder (combinational)
3	`sy_ready` = 1; write committed or `sy_rdata` valid; `mx_ready` = 1	Slave, fabric
4	Master in `M_DONE`: `req`, `valid` low; `done` = 1; `rdata_q` holds read data	Master
5	`mx_gnt` = 0 (or the other master's grant = 1)	Arbiter

Total: 4 clocks request to done, 5 clocks until the bus is free. The second master's grant on handover is also in cycle 5.

9. Timing diagrams

These diagrams are the cycle-accurate reference for RTL and testbench checks. Each cell is one clock cycle (value stable between rising edges). `I` is the idle cycle before the request. `‾‾‾‾` is high, `____` is low, `----` is don't-care. The report SHOULD redraw them as proper waveforms from simulation output.

9.1 Single write, M0 to S0 (address `0x0005`, data `0x0A`)

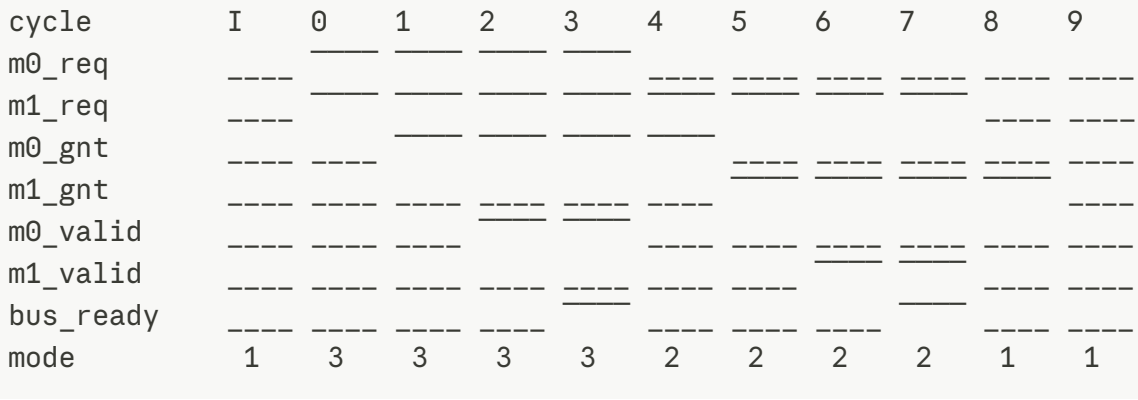
```
cycle        I    0    1    2    3    4    5
m0_req       ____ ‾‾‾‾ ‾‾‾‾ ‾‾‾‾ ‾‾‾‾ ____ ____
m0_gnt       ____ ____ ____ ‾‾‾‾ ‾‾‾‾ ‾‾‾‾ ____
m0_valid     ____ ____ ____ ‾‾‾‾ ‾‾‾‾ ____ ____
m0_rw        ____ ____ ____ ‾‾‾‾ ‾‾‾‾ ____ ____
m0_addr      ---- ---- ---- 0005 0005 ---- ----
m0_wdata     ---- ---- ----  0A   0A  ---- ----
sel_s0       ____ ____ ____ ‾‾‾‾ ‾‾‾‾ ____ ____
s0_ready     ____ ____ ____ ____ ____ ____ ____
m0_ready     ____ ____ ____ ____ ‾‾‾‾ ____ ____
m0_done      ____ ____ ____ ____ ____ ‾‾‾‾ ____
mode          1    2    2    2    2    1    1
```

The slave commits the write at the edge that starts cycle 3, when it samples `sel_s0 = 1` and `ready = 0`. `mode` is 2 while the single request is high.

9.2 Single read, M0 from S1 (address `0x1003`, stored value `0x5C`)

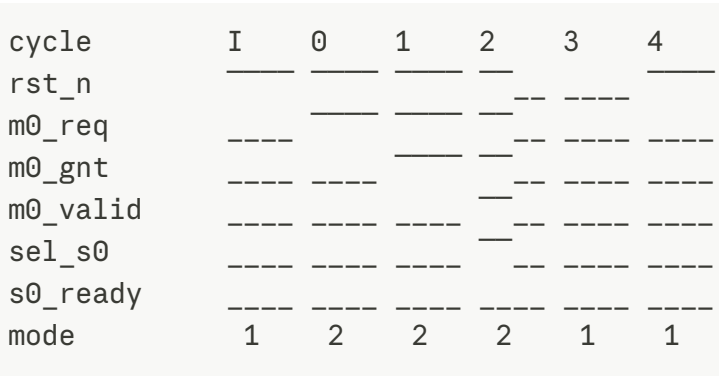
```
cycle        I    0    1    2    3    4    5
m0_req       ____ ‾‾‾‾ ‾‾‾‾ ‾‾‾‾ ‾‾‾‾ ____ ____
m0_gnt       ____ ____ ____ ‾‾‾‾ ‾‾‾‾ ‾‾‾‾ ____
m0_valid     ____ ____ ____ ____ ____ ____ ____
m0_rw        ____ ____ ____ ____ ____ ____ ____
m0_addr      ---- ---- ---- 1003 1003 ---- ----
sel_s1       ____ ____ ____ ‾‾‾‾ ‾‾‾‾ ____ ____
s1_ready     ____ ____ ____ ____ ____ ____ ____
m0_rdata     ---- ---- ---- ---- 5C   ---- ----   (valid only with m0_ready)
m0_done      ____ ____ ____ ____ ____ ‾‾‾‾ ____
rdata_q      ---- ---- ---- ---- ---- 5C   5C     (captured at the edge
starting cycle 4)
```

9.3 Two masters request in the same cycle (M0 wins, M1 waits, then handover)



M1 waits 5 cycles longer than it would on an idle bus. The grant moves from M0 to M1 at cycle 5 with no idle cycle in between, and the two grants are never high together. 7-segment shows 3 while both request, then 2 while only M1 still requests.

9.4 Asynchronous reset during a transfer



Reset asserts mid-way through cycle 2 (`‾‾__`). Grant, request and valid drop at once without waiting for a clock edge. The slave never samples `sel = 1` at the next edge, so the write is **not** committed. If instead reset asserts after the commit edge (during cycle 3 or later), the memory content stays, because `rst_n` does not clear the RAM (Section 7.2). Both cases are tests (TOP-07).

9.5 Unmapped address

Timing matches 9.1 with `sel_s0..2` all low and `decode_err` high during cycles 2 and 3. The responder registers `unmap_ready` in cycle 3, so `bus_ready` and `bus_err` are high in cycle 3, `bus_rdata` is `0x00`, no slave memory changes, and the master reaches `M_DONE` in cycle 4 with `err_q = 1`.

10. DE0 board implementation

Target device on both boards: Altera Cyclone III EP3C16F484 on the DE0, with the on-board 50 MHz oscillator, 10 slide switches, 3 push-buttons, 10 green LEDs, four 7-segment digits and two 40-pin expansion headers. Pin names below follow the DE0 User Manual naming; take exact pin locations from its pin-assignment file and do not copy them from this document.

10.1 Board partition

Board	Top module	Contents
A (bus)	`bus_top_a`	`bus_core`, reset synchroniser, 10 MHz bus-clock generator, page-switch shim, mode 7-segment driver, status LEDs
B (models)	`bus_top_b`	`master_model` x2 (`HOLD = 1`), `slave_mem` x3 and their read-data multiplexer, input synchronisers, address/data/request controls, read-data LEDs

10.2 Controls and display

Function	DE0 resource	Board	Behaviour
Reset	`BUTTON[0]`	A	Active low. Drives `rst_n` on A and, over the link, on B
Mode display	`HEX0_D`	A	Shows 1, 2 or 3 (Section 10.4). `HEX1_D` to `HEX3_D` blank
M0 request	`BUTTON[1]`	B	Held down: M0 asserts `req` and performs one transfer, then holds `req` until release
M1 request	`BUTTON[2]`	B	Same for M1
Address[3:0]	`SW[3:0]`	B	`cmd_addr[3:0]`; address bits [15:4] are 0 on board B
Data[3:0]	`SW[7:4]`	B	`cmd_wdata[3:0]`; bits [7:4] are 0

Function	DE0 resource	Board	Behaviour
Read/write	`SW[8]`	B	1 = write, 0 = read. [ASSUMPTION] not in the brief, needed to show a read
Slave page	`SW[1:0]`	A	Forced onto `bus_addr[13:12]`: 00 = S0, 01 = S1, 10 = S2, 11 = unmapped. [ASSUMPTION] extension so the demo reaches all three slaves and the error path
Bus status LEDs	`LEDG[0]` m0_gnt, `[1]` m1_gnt, `[2]` bus_ready, `[3]` decode_err, `[4]` sel_s0, `[5]` sel_s1, `[6]` sel_s2	A	Visible grant and select state
Read data LEDs	`LEDG[7:0]` last read data; `LEDG[8]` M0 busy; `LEDG[9]` M1 busy	B	Visible result

The push-buttons are hardware-debounced on the DE0 (the board manual describes a Schmitt-trigger circuit) and the slide switches are not, so both go through 2-flop synchronisers. Switch values are latched when the master starts a transfer, so changing a switch afterwards does not alter a transfer already in progress.

10.3 Inter-board link, clock and pin budget

The two boards each have their own 50 MHz oscillator, which must NOT both clock the design, because their frequencies drift apart. Board A therefore generates a single 10 MHz bus clock from its oscillator (PLL) and sends it to board B on an expansion-header clock pin, and board B uses that pin as its only clock. The 100 ns period leaves a wide margin for cable and pad delays. [ASSUMPTION] Link signals are then synchronous and are sampled directly, with no extra synchroniser stage, so the cycle counts in Section 8.5 hold across the boards.

A full 16-bit address plus 8-bit data on every lane would need far more than the 72 header I/O pins (2 headers x 36 [verify in the manual]). The demo link therefore carries a reduced address and the full 8-bit data lanes:

Link group	Direction	Signals	Pins
M0 request	B to A	`m0_req`, `m0_valid`, `m0_rw`, `m0_addr[3:0]`, `m0_wdata[7:0]`	15
M1 request	B to A	same as M0	15
Grants and ready	A to B	`m0_gnt`, `m1_gnt`, `m0_ready`, `m1_ready`	4
Return path	A to B	`bus_rdata[7:0]`, `bus_err`	9
Slave drive	A to B	`sel_s0..2`, `bus_rw`, `bus_addr[3:0]`, `bus_wdata[7:0]`	16
Slave return	B to A	`cl_rdata[7:0]` (read mux on B), `cl_ready` (OR of the three)	9
Reset	A to B	`rst_n`	1
Clock	A to B	`clk_bus`	1
Total			**70**

The wrappers pad the link to the core widths: board A zero-extends `mX_addr[3:0]` to 16 bits and ORs in the page bits at `[13:12]`; board B zero-extends the 4-bit slave address to 12 bits. Board A feeds `cl_rdata` and `cl_ready` back into all three slave ports of `bus_core`; that is correct because only the selected slave's value is used and the ready signals are ORed. The full-width `bus_core` is what the testbenches verify.

10.4 7-segment mode encoding

The DE0 segments are active low. Bit order `{g, f, e, d, c, b, a}`.

`mode`	Meaning	Digit	`HEX0_D[6:0]`
`2'd1`	Idle, no request	1	`7'b1111001`
`2'd2`	One master requesting	2	`7'b0100100`
`2'd3`	Two masters requesting	3	`7'b0110000`
`2'd0`	Illegal (not reachable)	-	`7'b0111111`

10.5 Demo script (what the marker will see)

Step	Action	Expected result
1. Reset	Press and release `BUTTON[0]` on A	`HEX0` = 1; LEDs for grants and selects off
2. Idle	Touch nothing	`HEX0` stays 1
3. One master write	B: `SW[8]` = 1, `SW[3:0]` = `0101`, `SW[7:4]` = `1010`. Hold `BUTTON[1]`	`HEX0` = 2; A `LEDG[0]` on, `LEDG[4]` pulses; B `LEDG[8]` on
4. One master read	`SW[8]` = 0, hold `BUTTON[1]`	B `LEDG[7:0]` = `00001010` (the data written)
5. Two masters	Hold `BUTTON[1]`, then also hold `BUTTON[2]`	`HEX0` = 3; A `LEDG[0]` on, `LEDG[1]` off (M1 waiting). Release `BUTTON[1]` and `LEDG[1]` turns on, `HEX0` = 2; release `BUTTON[2]` and `HEX0` = 1
6. Other slaves	A: `SW[1:0]` = 01, then 10, repeating steps 3 and 4	`LEDG[5]` then `LEDG[6]` pulse; read-back matches
7. Unmapped	A: `SW[1:0]` = 11, hold `BUTTON[1]`	A `LEDG[3]` (`decode_err`) on, no select LED; the master still finishes

11. Verification plan

Verification has three levels that match the brief's four task notes: arbiter (idle, one request, two requests), address decoder (3 slaves and the map), and top level (reset, idle, one master, two masters). All testbenches are self-checking: they compare against a reference model, count errors, and print a single `PASS` or `FAIL` line at the end.

11.1 Environment and conventions

- **Simulator:** ModelSim (Intel/Altera edition) or any SystemVerilog-capable simulator; Quartus for synthesis, fitting and TimeQuest.
- **Clock and reset in simulation:** `clk` = 20 ns period (50 MHz); `rst_n` low for the first 5 cycles unless the test says otherwise.

- **Drive and sample:** drive stimulus 1 ns after the rising edge; sample at the rising edge. This avoids races with the DUT's registered outputs.
- **Determinism:** random tests take a seed from a plusarg (`+seed=N`), print it, and the regression runs at least 10 seeds.
- **Timeout:** every test has a watchdog (default 2,000 cycles) so a hung handshake fails the test instead of hanging the simulator.
- **Logging:** each check failure prints time, check ID, expected and actual values, and increments a global error counter.

11.2 Testbench structure

Testbench	DUT	Contents
`tb_arbiter`	`bus_arbiter`	Request driver, cycle-accurate reference model of the Section 5.2 state table, per-cycle comparison of both grants, assertions CHK-01 to CHK-04
`tb_decoder`	`addr_decoder`	Directed vectors from Section 6.3, exhaustive 65,536-address sweep against a reference function, assertions CHK-08 and CHK-09
`tb_bus_core`	`bus_core`	Two behavioural master tasks (obey Section 4.4), three `slave_mem` instances, a shadow memory in the scoreboard, all checkers CHK-01 to CHK-11
`tb_demo_wrappers` (optional)	`bus_top_a` + `bus_top_b`	Both wrappers wired back to back, button and switch scripts, 7-segment decode check

11.3 Protocol checkers (assertions, active in every test)

ID	Check	Requirement
CHK-01	`m0_gnt` and `m1_gnt` are never both 1	REQ-SYS-03, REQ-ARB-05

ID	Check	Requirement
CHK-02	A grant never rises without the matching request high in the previous cycle	REQ-ARB-02
CHK-03	A grant is never removed while the owner's request is still high (no preemption)	REQ-ARB-03
CHK-04	When both requests are low in a cycle, both grants are low in the next cycle	REQ-ARB-05
CHK-05	`mx_valid` is high only while `mx_gnt` is high; `mx_req` stays high from request until `mx_ready`	Section 4.4 rules 2, 6
CHK-06	`rw`, `addr` and `wdata` are stable while `valid` is high until `ready`	REQ-BUS-01
CHK-07	`sy_ready` is a 1-cycle pulse that follows `sy_sel` by exactly 1 cycle; no `ready` without a prior `sel`	REQ-BUS-02
CHK-08	At most one of `sel_s0..2` is high; `decode_err` is never high together with a select	REQ-DEC-01, REQ-DEC-02
CHK-09	All selects and `decode_err` are low when `bus_valid` is low	REQ-DEC-03
CHK-10	A write changes only the addressed location in the addressed slave (shadow-memory compare after every transfer)	REQ-BUS-03
CHK-11	No `X` or `Z` on any `bus_core` output after reset is released	REQ-SYS-04

11.4 Test cases

Arbiter (`tb_arbiter`)

ID	Scenario	Stimulus	Expected	Covers
ARB-01	Reset	Power up with `rst_n = 0`, requests high, then release	Grants 0 during reset; state IDLE after release	REQ-SYS-02, REQ-SYS-04

ID	Scenario	Stimulus	Expected	Covers
ARB-02	Idle	Both requests low for 100 cycles	Both grants 0 throughout; `mode` = 1	REQ-ARB-05
ARB-03	M0 only	`m0_req` high for 6 cycles, then low	`m0_gnt` rises 1 cycle after `m0_req`, falls 1 cycle after `m0_req` falls; `mode` = 2	REQ-ARB-02
ARB-04	M1 only	Same as ARB-03 on `m1_req`	Same timing on `m1_gnt`	REQ-ARB-02
ARB-05	Both from idle	Both requests rise in the same cycle	`m0_gnt` = 1, `m1_gnt` = 0; `mode` = 3	REQ-ARB-01
ARB-06	No preemption	M1 granted, then M0 requests during M1′s grant	`m1_gnt` stays 1 until `m1_req` falls; `m0_gnt` stays 0 meanwhile	REQ-ARB-03
ARB-07	Handover M0 to M1	Both request, M0 releases first	`m1_gnt` rises in the same cycle `m0_gnt` falls; no cycle with both 1	REQ-ARB-04
ARB-08	Handover M1 to M0	M1 granted, M0 requests, M1 releases	`m0_gnt` rises in the cycle `m1_gnt` falls	REQ-ARB-04
ARB-09	Async reset in G0 and G1	Assert `rst_n` low mid-cycle while each grant is high	Grant falls within the same cycle (before the next edge); IDLE	REQ-SYS-02
ARB-10	Constrained random	100,000 cycles, random `req` patterns with masters honouring rule 7	Output equals the reference model every cycle; CHK-01 to CHK-04 silent	REQ-SYS-03, REQ-ARB-05

Address decoder (`tb_decoder`)

ID	Scenario	Stimulus	Expected	Covers
DEC-01	S0 boundaries	`0x0000`, `0x0001`, `0x0FFE`, `0x0FFF` with `bus_valid` = 1	`sel_s0` only	REQ-DEC-01

ID	Scenario	Stimulus	Expected	Covers
DEC-02	S1 boundaries	`0x1000`, `0x1001`, `0x1FFE`, `0x1FFF`	`sel_s1` only	REQ-DEC-01
DEC-03	S2 boundaries	`0x2000`, `0x2001`, `0x27FE`, `0x27FF`	`sel_s2` only	REQ-DEC-01
DEC-04	Unmapped	`0x2800`, `0x2FFF`, `0x3000`, `0x4000`, `0x8000`, `0xFFFF`	No select; `decode_err` = 1	REQ-DEC-02
DEC-05	Valid gating	Every vector of DEC-01 to DEC-04 with `bus_valid` = 0	All outputs 0	REQ-DEC-03
DEC-06	Exhaustive sweep	All 65,536 addresses, `bus_valid` = 1	Match the reference function; exactly one of 4 outputs high; counts: 4096, 4096, 2048, 55296	REQ-DEC-01, REQ-DEC-02

Top level (`tb_bus_core`)

ID	Scenario	Stimulus	Expected	Covers
TOP-01	Reset test	Hold reset, drive random master activity, release	All outputs idle during reset; first transfer after release succeeds; CHK-11 clean	REQ-SYS-02, REQ-SYS-04
TOP-02	Idle condition	No request for 1,000 cycles	No grant, no select, no `ready`; memories unchanged; `mode` = 1	REQ-SYS-04
TOP-03	One master request	For each of M0 and M1: write then read-back at the first, last and a middle address of S0, S1 and S2, with data `0x00`, `0xFF`, `0x55`, `0xAA`	Read-back equals written value; timing of Section 8.5; `ready` after exactly 1 wait state	REQ-SYS-01, REQ-BUS-01, REQ-BUS-02, REQ-BUS-04

ID	Scenario	Stimulus	Expected	Covers
TOP-04	Two master requests	(a) both in the same cycle; (b) M1 first, M0 joins during M1's transfer; (c) M0 first, M1 joins; (d) both repeat 50 times back to back	M0 wins (a); no preemption (b, c); each transfer completes with correct data; wait times match Section 9.3; CHK-01 to CHK-06 silent	REQ-SYS-03, REQ-ARB-01, REQ-ARB-03
TOP-05	Unmapped access	Read and write `0x2800` and `0xFFFF` from each master	Transfer completes with `mx_err` = 1; read data `0x00`; no memory changes	REQ-BUS-05
TOP-06	Back-to-back stress	10,000 random transfers from both masters, random addresses (80 % mapped)	Shadow memory matches after every read; zero checker failures	REQ-BUS-03, REQ-BUS-04
TOP-07	Reset mid-transaction	(a) assert reset before the slave commit edge of a write; (b) assert reset after it	(a) memory location unchanged; (b) memory holds the new value; in both, the bus is idle after release	REQ-SYS-02
TOP-08	Memory isolation	Fill each slave with a distinct pattern, then write single locations and re-read all three slaves	Only the targeted location changed	REQ-BUS-03, REQ-BUS-04
TOP-09	Hardware demo	Follow the Section 10.5 script on two DE0 boards	Every step matches; `REQ-DEMO-01` to `03` observed	REQ-DEMO-01 to 03

11.5 Coverage goals and exit criteria

- **Code coverage:** 100 % statement, branch and FSM state/transition coverage on `bus_arbiter`, `addr_decoder` and `master_model`; 100 % statement and branch on `bus_core` glue. Any exclusion needs a written reason.
- **Functional coverage bins:**
  - arbiter: each of the 9 rows of the Section 5.2 state table;
  - decoder: each region start, end and one interior address, plus 3 unmapped regions;
  - top level: master x slave x read/write (12 bins) plus unmapped x master x read/write (4 bins);
  - reset asserted in each of IDLE, G0 and G1, and in each master state.
- **Exit criteria:** (1) every REQ-xx in Section 3 has at least one passing test; (2) the full regression passes on 10 seeds with zero checker failures; (3) coverage goals above are met; (4) Quartus compile has no inferred latches, and the timing report meets REQ-NF-01; (5) TOP-09 is signed off on hardware.
- **Bug reporting:** a failing test is logged with test ID, seed, simulation time and the failing check ID, so the designer can reproduce it with one command.

12. RTL guidelines, deliverables and schedule

12.1 RTL coding rules

- One module per file; file name equals module name. Start each file with a header naming the block, its REQ-xx IDs and the Section number of this spec.
- Every flop uses the asynchronous-reset, positive-edge template below, with non-blocking assignments. Combinational blocks use `always @*` with blocking assignments and a default value for every output, so no latch is inferred.
- No gated or derived clocks inside the core; the only clock generation is the PLL in `bus_top_a`. No delays (`#`) in synthesizable code. No `initial` blocks except the RAM initialisation in `slave_mem`.
- Widths and sizes come from `parameter` or `localparam`; no magic numbers in the decode logic.
- State encoding is declared with `localparam`; the FSM has a `default` branch that returns to IDLE.
- Resets and asynchronous inputs: use `reset_sync.v` and `sync_2ff.v`; never use a raw button or switch in logic.

- Code must pass the simulator's lint or Quartus analysis with zero warnings about latches, unconnected ports or width mismatches.

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) q <= {W{1'b0}};
    else        q <= d;
end
```

12.2 Repository layout

```
rtl/     bus_arbiter.v  addr_decoder.v  bus_core.v  master_model.v
slave_mem.v
         bus_top_a.v  bus_top_b.v  reset_sync.v  sync_2ff.v  seg7_mode.v
tb/      tb_arbiter.sv  tb_decoder.sv  tb_bus_core.sv  bfm_master.sv
checkers.sv
sim/     run scripts, wave setup, seed list, coverage reports
quartus/ board A project, board B project, pin assignments, timing
constraints (SDC)
doc/     this specification, the report, waveform exports
```

12.3 Deliverables

Deliverable	Marks	Content, taken from the brief	Spec source
Report	20	Functional descriptions	Sections 2, 5 to 8
		IO definitions	Section 4 tables, 5.1, 6.1, 7.1, 7.2, 8.2
		Timing diagrams (from real simulation waveforms)	Section 9 as the reference
		RTL code and testbench code	Appendix of the report
Demo	20	Working two-board system following the script	Section 10.5

12.4 Schedule (15 days)

The brief gives a 15-day deadline. [ASSUMPTION] Day 1 is today, 21 Oct 2026, so the deadline is 5 Nov 2026; correct this if the brief was issued on another date.

Days	Dates	Work	Exit gate
1 to 2	21 to 22 Oct	Review and freeze this spec; resolve the Section 13 open issues; set up repository	Spec signed off by developers and testers
3 to 5	23 to 25 Oct	`bus_arbiter`, `addr_decoder` and their testbenches	ARB-01 to ARB-10 and DEC-01 to DEC-06 pass
6 to 8	26 to 28 Oct	`master_model`, `slave_mem`, `bus_core`, `tb_bus_core`	TOP-01 to TOP-08 pass; coverage goals met
9 to 10	29 to 30 Oct	Board wrappers, PLL and clock link, pin assignments, timing constraints, synthesis of both projects	Both projects fit; REQ-NF-01 met
11 to 12	31 Oct to 1 Nov	Two-board bring-up and demo rehearsal	TOP-09 passes on hardware
13 to 14	2 to 3 Nov	Report: diagrams from waveforms, code appendix, review	Report reviewed by a second person
15	4 to 5 Nov	Final regression, submission, demo	Submitted before the deadline

13. Assumptions, open issues and glossary

The brief leaves 13 design choices open. Each is listed here with what changes if the instructor decides otherwise. Review and sign these off before RTL coding starts.

13.1 Assumptions

ID	Assumption	Where used	If wrong
A1	Address bus is 16 bits wide	2.2, 6	Change `ADDR_W`; decoder constants shift
A2	M0 has priority over M1	5	Swap the two priority branches in the IDLE row; update ARB-05

ID	Assumption	Where used	If wrong
A3	Arbitration is non-preemptive (grant held until the owner drops `req`)	5.3	Arbiter FSM and CHK-03 change
A4	Reset asserts asynchronously, releases synchronously	2.2, 8.4	Wrapper change only
A5	One transfer per grant in the baseline; slave has exactly one wait state	4.4, 7.2	Timing tables in 8.5 and 9 change
A6	`rst_n` does not erase slave memory (block RAM)	7.2	Needs register-based memory (too large) or a clear sweep after reset
A7	Unmapped accesses complete with an error flag and return `0x00`	7.3	TOP-05 and REQ-BUS-05 change
A8	Demo adds a read/write switch (`SW[8]`), a slave-page switch pair on board A, and a request-hold behaviour in the demo masters	10.2	Demo script changes
A9	The demo bus clock is 10 MHz, made on board A and shared with board B	10.3	Link timing re-analysis
A10	The link carries a 4-bit address and 8-bit data lanes because of the header pin count	10.3	Wider address needs a time-multiplexed or reduced link
A11	The "Assignment 3 - Guidelines" slide belongs to this assignment	1.3	Requirements REQ-DEMO change
A12	Day 1 of the 15-day schedule is 21 Oct 2026	12.4	Shift all dates
A13	Memory map: S0 `0x0000`, S1 `0x1000`, S2 `0x2000` (the brief gives sizes only)	2.3	Decoder constants and test vectors change

13.2 Questions for the instructor

1. Which master should have the higher priority, or does the choice not matter (A2)?
2. Is a 4-bit address on the inter-board link acceptable for the demo, given the 4 address switches in the guidelines (A10)?
3. May the two boards share a slowed 10 MHz clock instead of running at 50 MHz (A9)?
4. Should the 7-segment mode follow the request lines (our reading) or a user-selected mode?
5. Is the guidelines slide labelled "Assignment 3" a typo (A11)?

13.3 Risks

Risk	Likelihood	Mitigation
Hold-time violation on board A to B link signals at the same clock edge, because board B's clock arrives later	Medium	Constrain the link in TimeQuest; if it fails, add a PLL phase shift on board B or insert delay on the capture path
Header pin budget is tight (70 of 72)	Medium	Freeze the link table early; drop `bus_err` or `m1_ready` pins first if a pin is lost
Block RAM not inferred (slaves built from logic cells)	Low	Use the vendor RAM template; check the fitter's memory report
Master holds `req` forever and starves the other	Low (demo only)	`HOLD` mode releases on button release; protocol rule 7 in all other cases
Spec drift between report and RTL	Medium	Every REQ-xx traced to a test; changes to this spec go through the same review as RTL

13.4 Glossary

Term	Meaning
Master	Block that starts transfers (M0, M1)
Slave	Block that answers transfers (S0, S1, S2)

Term	Meaning
Grant / request	Arbiter's permission to use the bus / a master's ask for it
Handover	Direct grant move from one master to the other with no idle cycle
Wait state	Extra clock a slave takes before asserting `ready`
Moore FSM	State machine whose outputs depend only on the current state
One-hot	Encoding in which exactly one bit is 1
BFM	Bus-functional model: a testbench task that drives a bus port
M9K	Cyclone III embedded 9 Kbit memory block
MIF	Memory initialisation file
SDC	Synopsys design constraints file used by TimeQuest